

The Aha Moment: Keyi, a Multimodal Learning Experience Builder that Installs Insight, Not Content

Chongyang Wang
Northeastern University
Boston, USA
wang.chong@northeastern.edu

Vedant Shah
Northeastern University
Boston, USA
shahh7107@gmail.com

Sri Sai Teja Mettu Srinivas
Northeastern University
Boston, USA
srisaitejms2002@gmail.com

This course project report uses the ACM two-column style rather than the course template for three reasons: the first author is a PhD student who plans to extend this work into a full research project, the reporting guidance arrived too close to the deadline for a rewrite, and the ACM structure keeps every required item easy to locate. For the required components: problem definition and GenAI fit (Sections 1 and 2); baseline comparison (Section 2 and Table 1); application and technical depth of GenAI techniques (Section 3); data, knowledge sources, and dataset quality (Sections 3.3 and 5); evaluation pipeline (Section 5); experiments performed (Sections 4 and 6); visualizations and tabular representations (figures and tables throughout); codebase links (Appendix A). One deliberate departure: the report keeps an in-progress iteration history (Section 4) recording the problems we ran into, how we solved them, and what we learned. We also provide an interactive visualization of the entire agentic pipeline at <https://github.com/GaryTheKid/3d-vr-codeless-platform/blob/main/agent-pipeline.html>; download it and double-click to open it in a local browser.

Abstract

AI-generated courses tend to look great and teach nothing. Generative AI has made it effortless to turn a document into something that looks like a course, but not into something that teaches: evaluations keep finding task performance without durable learning, modality choices follow whatever is cheap to generate, nothing guarantees that the generated material covers the intended learning goals, and learners are left to wander through the output without guidance. We built Keyi because we hit this wall ourselves: our early prototypes produced impressive interactive scenes that, once the visual novelty wore off, taught nothing. Keyi is a browser-based agent that compiles teaching material, or a short conversation with the author, into a complete multimodal learning experience: readings, 2D interactives, quizzes, and 3D scenes, organized the way an excellent teacher prepares a lesson rather than the way a slide generator dumps content. The agent first distills the small set of “aha” insights a learner must walk away with, anchors them in a machine-checkable knowledge structure, and only then produces content, with every section accountable to the goals it claims to teach. A learning companion then guides learners through the result step by step. Under the hood, a planning and execution agent architecture with 20 dynamically loaded skills and 40 tools keeps generation reliable. We evaluated the produced learning experiences in a remote usability study across five subject packs ($n = 6$), with pre and post quizzes, workload and satisfaction questionnaires, and interviews. Mean quiz totals rose from 1.5 to 23.8 of 30 points, satisfaction averaged 6.4 of 7, and participants consistently named the interactive and 3D sections plus the quizzes as the learning engine, while thin reading scaffolds and occasional 3D fidelity issues emerged as the clearest frictions. We close with what the study supports, what it challenges, and the iteration history that shaped the system.

Keywords

Learning agents; course generation; knowledge graphs; multimodal learning; human-AI co-authoring; usability study

1 Introduction

1.1 Problem

Authoring an interactive learning experience has always demanded three scarce skills at once: programming, interaction design, and pedagogy. Generative AI removed much of the first cost and some of the second, and a wave of LLM-powered learning products followed. Yet when these systems are asked to “make a course from this PDF,” what they overwhelmingly produce is a repackaging of the source: tidy sections, bullet lists, stock visuals, and the occasional quiz, arranged in page order. The output looks complete. Whether it teaches is another question.

We think the field’s current generation of LLM learning agents falls short in four connected ways. First, *measured learning gains are rare*. The OECD’s 2026 outlook on generative AI in education reports the pattern bluntly: students using generative AI completed tasks far more successfully, yet performed *worse* once the AI was taken away, because finishing a task with an AI tool is not the same as learning from it [22]. A large field experiment behind that finding showed unrestricted GPT-4 help lifting practice performance while harming exam performance, with guardrails eliminating the harm but producing no gain [2]; letting the AI replace the learner’s cognitive process damages learning in both the short and the long run. Engagement metrics improve; learning often does not. Second, *modality choice has no principled basis*. Which medium serves a concept is a decades-old research question with real answers [6, 15, 20], yet generated material defaults to whatever is cheap to produce rather than what the concept cognitively needs. Third, *pedagogical coverage is not ensured*. Without an explicit representation of what must be taught, generated units drift from the stated goal, skip prerequisites, and assess concepts that were never introduced; models exhibit exactly this gap between solving a problem and teaching it [32]. Fourth, *learners are left unguided*. A pile of generated material carries no enforced order, no staging from scaffolding through practice to transfer [34, 36], and no companion that knows where the learner currently is and what they just got wrong. We treat this characterization as the design gap our system answers.

We did not arrive at this critique from reading alone. Our own early prototype, a codeless builder for interactive 3D learning scenes, produced demos that looked impressive and failed the simplest litmus test: strip the visuals, and no teaching strategy remained (Section 4 tells that story). That moment reframed the entire project. The interesting problem is not generating learning content; it is generating *teaching*.

1.2 Approach

Keyi is a browser platform, requiring no build step and no coding background, that treats course generation the way an excellent teacher treats lesson preparation. We call the system Keyi after the key it forges for each aha insight and the Mandarin *keyi* (“can, to be able”): an installed insight is exactly what makes a learner able. Before producing any content, the agent distills the small set of transferable insights the learner must walk away with, maps the concepts and their prerequisite structure into a knowledge graph, and plans an outline in which every section explicitly declares which concepts it covers and which insights it installs. Only then is content generated, section by section, in the modality each concept needs: prose where prose suffices, 2D interactives for parameter exploration, quizzes for transfer, and 3D scenes only where the concept has intrinsic spatial structure. The author refines the result through natural language and direct manipulation; a learning companion then walks learners through the finished experience step by step. The same platform serves whoever is building: a teacher preparing material for a class, or a learner compiling a course for themselves.

1.3 Contributions

This report makes five contributions:

- (1) **A pedagogy-first compilation approach.** We turn four habits of excellent teachers into structure the system can check: distilling the aha insights a course must install, anticipating misconceptions, holding one running example, and staging learning from scaffold to transfer. Whether a course teaches becomes a property you can inspect, not a hope about the model.
- (2) **An agent designed for reliable generation.** A planning and execution split, dynamic skill loading, disciplined tool calling, and budget governance keep a long multi-step compilation dependable even when individual model calls are not.
- (3) **A complete multimodal learning experience builder.** Reading, 2D interactive, quiz, and 3D sections are authored, refined, and delivered in one browser environment, with a step-by-step learning companion on the delivery side.
- (4) **Usability evidence from a completed six-participant study.** A remote protocol, five subject packs, and instruments spanning learning gain, workload, satisfaction, and interviews, with six sessions completed and analyzed.
- (5) **A candid iteration history.** The path from a 3D spectacle generator to a pedagogy-driven builder, including the failures, is reported with the lessons we believe transfer.

The thesis underneath all of this is simple. LLMs default to PPT machines; excellent teaching installs keys. Keyi turns that difference

into enforceable data contracts and engineering contracts, rather than relying on model goodwill alone.

2 Related Work

Intelligent tutoring and authoring systems. Classical ITS research established dialogue tutoring, knowledge tracing, and adaptive questioning [7, 9, 25], with step-based tutors approaching the effectiveness of human tutors [33]. Our system differs in that it generates full multimodal course structures from source documents, not only adaptive question sequences.

LLM learning systems. The LLM era brought document-to-course generation and tutoring at scale: Google’s LearnLM line fine-tunes models for pedagogy and transforms textbooks into multiple representations [16, 17], Tutor CoPilot supports human tutors with real-time suggestions [35], and conversational tutoring systems wrap LLM generation in classical tutoring scripts [29] or steer it along predefined pedagogical plans [26]. Two lessons from this line shape our design. Carefully engineered, pedagogy-first AI tutoring can beat even active-learning classrooms [13], so the ceiling is real; but the same models unstructured can harm learning [2, 22], and the field is converging on putting explicit structure around the LLM rather than trusting prompts. Keyi belongs to that convergence, with the structure drawn from how excellent teachers prepare.

LLM agents and tool use. ReAct-style reasoning-and-acting loops [38] and learned tool invocation [28] shape our agent architecture. We adopt skill routing plus verbose tool documentation as a belt-and-suspenders strategy so that weaker models remain reliable.

Generative 3D authoring. Language-guided scene generation [37] and real-time prompting of interactive worlds [8] demonstrate chat-to-scene spectacle. Our HCI-adjacent piece is natural-language inspection: selecting an object surfaces capability cards written in plain language, instead of confining authors to code-level editing.

Learning sciences. Constructionism [23], the ICAP engagement hierarchy [5], retrieval practice [27], desirable difficulties [3], and multimedia modality principles [20] motivate our pedagogy. Rich 3D delivery specifically is known to add cognitive cost without adding learning when the concept does not need it [24], which is why our spatiality gate admits 3D only when the concept has intrinsic spatial structure.

Document and knowledge structure. PDF-to-structure conversion is mature tooling [1], and extracting concepts and prerequisite relations from text has a long history [18, 21], as does machine reasoning over knowledge graphs [11]. We run conversion locally and treat pedagogical figures as first-class syllabus sources.

Positioning. Prior systems tend to excel either at chat-to-scene spectacle or at plain text course generation, and the LLM learning agents deployed today inherit the weaknesses laid out in Section 1 [2, 22, 32]. Keyi targets the gap between these families: document-grounded, graph-checked, multimodal learning experiences with author-in-the-loop refinement and a companion-guided learning mode.

Table 1: The excellent-teacher versus PPT-machine contrast, and how each dimension is encoded in the system.

Dimension	PPT machine	Excellent teacher	Encoded as
Prep start	Page-order slides	2 to 5 must-get insights	Aha keys
Structure	Linear facts	Mental graph with break-points	Knowledge graph, coverage contracts
Misconceptions	Ignored	Anticipated	Misconception fields, quiz distractors
Abstraction	States the conclusion	Student operates it out	Build ideas, construction rules
Examples	New every slide	One running example	Anchor example
Figures	Decoration	Densest knowledge	Figure ground-ing
Assessment	Reskins numbers	Reskins the story	Transfer quiz rules
Modality	Whatever is handy	Cognitive need	Spatiality gate
Completeness	Talks until done	Closed loop	At least one reading and one quiz; every insight installed

3 System Design

3.1 Pedagogical Framework

3.1.1 Excellent teacher versus PPT machine. The design starts from a contrast between how an excellent teacher prepares and what a content dump produces. Table 1 summarizes the contrast and how each dimension is encoded as a checkable artifact in our pipeline.

3.1.2 The Aha moment. Each course distills two to five “aha keys”: the small set of transferable insights that, once truly installed, let a learner solve problems that merely look different on the surface. An aha key carries a short insight phrased so a student can rehearse it, the misconception it replaces, an explanation of why this key unlocks reskinned problems, and a build idea: a manipulable experience through which the learner can construct the insight rather than be told it. Aha keys are distilled *before* any outline or content exists; everything downstream is accountable to them.

3.1.3 Knowledge graph as a hard mid-anchor. Between the source material and the course outline sits a knowledge graph: the concepts to be taught, the prerequisite edges between them, the aha keys, a single anchor example that runs through the course, and the target level. The graph is not decorative; it is the contract that later stages are validated against. Every concept a section claims to cover must exist in the graph; teaching order must respect prerequisite edges; quizzes may only probe concepts that were taught; every aha key must be installed by at least one section, with a fallback that assigns orphaned keys to the most suitable interactive section; and

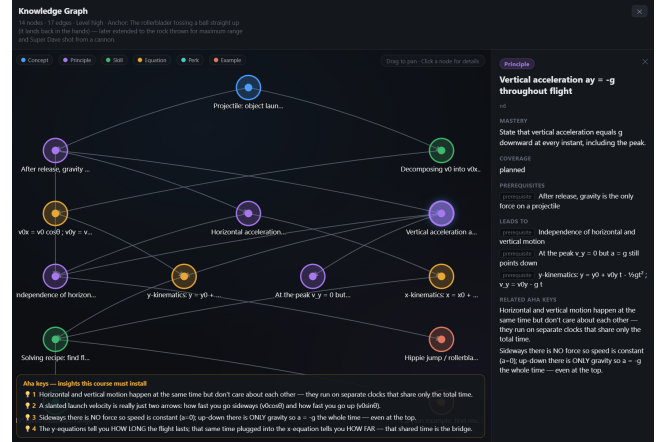


Figure 1: A generated knowledge graph as shown in the platform’s graph viewer (projectile-motion sample course: 14 nodes, 17 edges). Node types are color-coded; selecting a node opens its mastery statement, coverage status, prerequisite edges, and related aha keys (right); the distilled aha keys the course must install are listed at the bottom.

the core figures of the source document must contribute concepts of their own. Coverage checking, orphan-key reassignment, and the minimum-section guarantees are enforced in the current version; a stricter critic that cross-checks the generated content itself against the graph is only partially wired (Section 8). Figure 1 shows a generated knowledge graph with its aha keys.

3.1.4 Scaffold, construct, transfer. Sections follow a three-stage pedagogy. *Scaffold* sections (reading) name the misconception, contrast it with reality, and state the insight in student language, following the refutation-text structure that meta-analytic evidence favors for misconception change [30]. *Construct* sections (2D interactive or 3D) run a predict, act, observe cycle whose outcome contradicts the misconception and asks the student to articulate what happened; they must never merely display the conclusion, in line with constructionist and ICAP accounts of why doing beats watching [5, 23]. *Transfer* sections (quiz) include at least one item per aha key wrapped in a new surface story, with distractors that embody the misconception; retrieval practice itself is one of the best-evidenced learning interventions available [27].

3.1.5 Modality economics. 3D is the costliest modality, both to generate and cognitively [24, 31], so it is admitted only when the concept has intrinsic spatial structure or dynamics: molecular geometry, terrain, projectile paths, gear meshes. 2D parameter exploration goes to interactive pages, prose goes to reading, and mastery checks go to quizzes. Decorative 3D is treated as a defect, not a bonus.

3.2 HCI Principles

Several interaction principles are baked into the agent rather than left to convention.

- **The author stays in control.** Complex plans require confirmation before execution, the agent’s thinking streams

visibly, and selecting a 3D object surfaces plain-language capability cards describing what that object can do.

- **Selection is context.** Multi-selecting objects pins them into the conversation; there is no separate attach ritual.
- **Controls are non-destructive.** One toggle changes one semantic property of an object, such as whether it spins in place, and never silently replaces neighboring behavior.
- **The platform owns feedback; content owns results.** Hover and click highlights are platform behavior, what happens after an interaction is course content, and the two meet at a small set of semantic events (activate, grab, drag, release).
- **Determinism follows error cost.** Creative work is generated as code; structural work comes from deterministic builders (Section 3.3).
- **Bilingual by construction.** Generated content follows the interface language, not the language of the source material.

3.3 System Pipeline

Figure 2 shows the pipeline end to end, including the turn routing around it; this section walks through its stages.

3.3.1 Parsing the input. A course starts from a PDF or Word document, or simply from a conversation with the author. Uploaded documents are converted locally by a document-understanding service (Docling [1]) into markdown plus extracted figures. Figures then pass a two-step grounding: a heuristic filter drops obvious noise such as logos and page furniture, and a vision model examines the largest survivors, classifying each as core, supporting, decorative, or noise, and writing a short visual summary of what the figure teaches. Core figures are treated as first-class sources: the planner is required to extract concepts from them, not just from the text. To keep a long generation honest, the identity of the source document is locked into every downstream prompt, so an upload made mid-run cannot silently retarget the course being built.

3.3.2 The course data model. Everything the pipeline produces lives in one portable course document. A course holds its outline: chapters, and within them sections, where each section has a type (reading, 2D interactive, quiz, or 3D), a stated purpose, the list of concepts it covers, the list of aha keys it installs, a build status that drives the interface’s progress indicators, and the content itself. Alongside the outline, the course stores the knowledge graph itself and a snapshot of the current 3D scene. The coverage and installation lists are the load-bearing part: they are what make the pedagogy checkable, because every section publicly declares what it is for, and the validation rules of Section 3.1 can be enforced mechanically against these declarations.

3.3.3 Planning: aha keys and the knowledge graph. A single planning call reads the converted material and the figure summaries and produces, in order, the aha keys, the knowledge graph, and the chapter and section plan with each section’s coverage and installation declarations. The strict ordering matters: insights first, structure second, outline third, never the reverse. The planning output is parsed defensively, with retries and repair for the ways model output goes wrong in practice, and all author-facing text follows the interface language.

3.3.4 Populating the outline. A normalization pass then applies the validation rules: orphaned aha keys are reassigned, every course is guaranteed at least one reading and one quiz section, and coverage declarations are checked against the graph. Section content is then generated in parallel, a few sections at a time, except that 3D sections fill strictly one after another for reasons explained below. Each section’s generation context carries the source lock, the concepts it must cover, the aha keys it must install, summaries of its sibling 3D scenes so that scenes stay distinct, up to six pedagogical figures, and the relevant slice of the source material. As sections complete, their rings in the outline turn green; when all sections are done, the learning mode unlocks.

3.3.5 Section modalities. Four modalities carry the pedagogy (Figure 3).

Reading. Four to ten short chunks of formatted prose. Figures from the source document are injected where they belong; when the platform runs with image generation available, sections aim to include at least one generated illustration. Sparse follow-up questions are embedded for the companion and quizzes to reuse.

2D interactive. The agent first decides what kind of interaction the concept needs (walking a process, varying a condition, matching, open exploration, or predict-then-check) and then generates a small self-contained interactive page, sandboxed from the rest of the platform. A generated page that turns out to be a static poster counts as a failure and is retried, because interactivity is the point.

Quiz. Two to five items with at least one short-answer question. Transfer rules require a fresh surface story per aha key, distractors that embody the documented misconception, and questions that only probe taught concepts. During learning, multiple-choice grades locally; short answers are graded by the model, which returns a judgment and feedback without revealing the full key, and wrong answers quietly notify the learning companion.

3D scenes. This is where we invested the most design effort, and the generation follows a pipeline of its own within the big pipeline:

- (1) *Requirement decomposition.* The section’s aha key, its build idea, and the covered concepts are decomposed into what the scene must let the learner do, not merely see.
- (2) *Object ontology.* The agent drafts the full set of objects the scene needs and the relationship network among them: which objects teach, which support, which respond to which interactions.
- (3) *Object generation.* Each object is created through the skill and tool machinery described in Section 3.4: creative objects are written as small Three.js programs in a sandbox, while structural elements such as rooms, stairs, and information panels come from deterministic parametric builders.
- (4) *Interaction wiring.* Objects are given behaviors against the platform’s semantic events, so that the predict, act, observe loop of the construct stage happens.
- (5) *Self-validation and debugging.* The agent inspects its own scene: are there enough teaching objects, is a panel present, is the scene distinct from its siblings, does everything survive a save and reload? It iterates until the scene passes, then marks the section done.

Three engineering commitments make this loop trustworthy. First, *one live scene*. The platform maintains exactly one live 3D

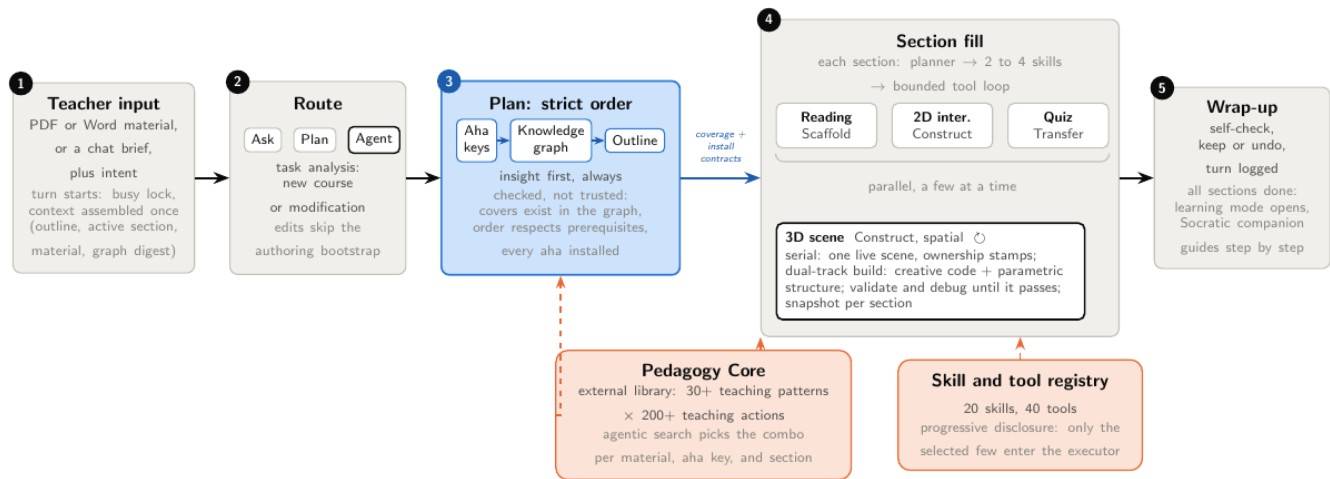


Figure 2: The Keyi agent pipeline. The horizontal spine follows one authoring turn: (1) teacher input and turn setup, (2) mode routing and task analysis, (3) the insight-first planning chain from aha keys through the knowledge graph to the outline, (4) section fill across the four modalities with the serial 3D build, and (5) wrap-up into guided learning. Dashed vertical arrows mark resource injections: the Pedagogy Core library and the skill and tool registry.

scene at a time; each 3D section keeps a serialized snapshot of its scene, captured through Three.js’s standard scene-graph serialization (ObjectLoader), which can rebuild the scene later from plain data. This is why 3D sections fill one at a time rather than in parallel. Second, *ownership and guards*. During a fill, every created object is stamped with the section that owns it, and objects from other sections are refused. Two rules protect saved work: an empty scene can never overwrite a non-empty snapshot, and a single corrupted object can never blank a section. Loading instead falls back through smaller salvage steps, down to regenerating an object from the code that first created it. Third, *a tracked sandbox*. Objects created by generated code are registered together with the code that produced them, so the platform can verify exactly what the sandbox added to the scene and can faithfully rebuild those objects after a reload.

3.4 Agent Design

The agent is organized around a strict separation between thinking about what to do and doing it.

The turn lifecycle. When the author sends a message, the platform assembles the full working context once, and locks it for the remainder of the turn, so the agent’s view of the world cannot drift mid-turn. A *Planner* call then classifies the request: its intent, its complexity, and which of the agent’s skills are relevant. Pure questions are answered directly with no tools. Simple requests go straight to execution. Complex requests produce a visible plan that the author confirms before anything changes, keeping the human in control of consequential edits. An *Executor* call then carries out the work through tool calls, up to twenty rounds per turn, and every successful change to the course is persisted immediately so that a crash mid-turn cannot lose completed work.

Dynamic skill loading. The agent has twenty skills, each a package of specialized instructions for one kind of work: organizing

scenes, creating objects, custom modeling, experiment logic, animation, information panels, pedagogy, validation, interaction design, navigation, room design, debugging, and the course pipeline stages themselves. The Executor never sees all twenty. The Planner selects the two to four skills the request needs, and only those instructions are loaded into the execution context. This progressive disclosure keeps the execution prompt small and focused, which is what keeps weaker models reliable.

Tool calling. The Executor acts on the world exclusively through forty tools, grouped into families for building, editing, panels, queries, environment, spatial layout, outline management, and course generation. Each tool carries deliberately thick documentation: not just a signature, but guidance on when to use it, when not to, and what mistakes look like. The tools also encode structural protections in themselves rather than trusting the model’s judgment: the agent cannot restructure the course outline unless the author explicitly asked for it, cannot create empty placeholder sections, and cannot switch the live 3D scene as a side effect of unrelated edits. In effect, the safety rules live in the tool layer, where they cannot be talked out of.

The agent loop, nested. The 3D generation pipeline of Section 3.3 runs as a nested agent loop of its own: a sub-agent with a reduced toolset of about ten scene tools, operating under the same plan, act, verify discipline, pinned to the one section it is allowed to touch. The same loop shape recurs at both scales, an outer loop for the author’s request and an inner loop for each 3D scene, which kept the architecture conceptually simple even as capabilities grew.

Managing context at scale. A busy scene can exceed what fits in a prompt. Up to twenty objects are described in full; beyond that, the agent sees a one-line index of the scene and pulls details on demand, helped by a cheap relevance ranking based on the current selection, recent edits, and text overlap with the request. Each object maintains a short natural-language description of its

(a) Reading · Scaffold

From Straight Lines to Curved Flight

Bridge from Chapter 3's linear motion and forces to launched objects, defining a projectile and its single acting force.

#1 From Chapter 3 to Launched Objects

A projectile follows a curved trajectory; its launch velocity v_0 has horizontal and vertical components

These launched objects are called **projectiles**, and they are the focus of this chapter.

(b) 2D interactive · Construct

Step 1 — Predict the outcome: The driver gear on the right looks about 4x bigger. If you turn the driver 1 full turn, how many turns does the driven gear make?

Your guess: Lock guess

Driver teeth: 10 Driven teeth: 40

Enter Count-teeth mode Turn driver +1 rotation Reset markers

Driver turns: 0 Driven turns: 0.00

Presets: 10 / 40 40 / 10 12 / 24 Trap

Driver teeth: 10 Driven teeth: 40

Step 4 — Reveal

Ratio = Driven ÷ Driver = 40 / 10 = 4 : 1

Driven turns **SLOWER (4.00x)** — speed decrease

Real compound gear train — ratio still = driven ÷ driver teeth.

(c) 3D scene · Construct

A compound train has stage ratios 2:1 and 4:1. Overall?

Tap an option below

A. 8:1 (you add them)

B. 8:1 (you multiply them)

C. 2:1 (only the first counts)

Compound Ratio

Stage 1: 20 / 10 = 2:1

Stage 2: 40 / 10 = 4:1

Overall = 2 x 4 = 8:1

Why Multiply?

Blue input: 10 teeth (driver)

Red big + Yellow small: fixed on one shaft

Green output: 40 teeth (driven)

Overall ratio: Stage1 x Stage2

One giant gear could NOT reach 8:1 alone

Your Task

1. Find the compound gear red 20t + yellow 10t on ONE shaft

2. Ratio of stage 1: driver/driver = 20/10 = 2:1

3. Ratio of stage 2: 40/10 = 4:1

4. Multiply them 2 x 4 = 8:1

5. Click & watch 8 input turns = 1 output turn

Two Gears on One Shaft

A COMPOUND gear = two gears fixed on ONE shaft, turning together.

Stage 1: input → big gear

Stage 2: small gear → output

Overall = Stage 1 × Stage 2

(d) Quiz · Transfer

Ratio Check

Verify learners compute two-gear ratios, predict direction, and understand idlers.

Q1

In a clock mechanism, a driver gear with 12 teeth meshes with a driven gear with 36 teeth. What is the gear ratio, and does the driven gear speed up or slow down?

3:1, driven turns slower

3:1, driven turns faster

1:3, driven turns faster

depends on which gear is larger in diameter, not the teeth

Q2

A bicycle mechanic has a driver gear of 15 teeth and a driven gear of 45 teeth, but the driver is bright red and the driven is dull gray. Some students say the shiny red gear 'looks stronger' so it sets a bigger ratio. Explain what actually determines the ratio and state it.

Type your answer here...

Submit

Figure 3: The four section modalities as delivered to learners, from the generated sample courses: (a) a reading section with an injected source figure (projectile motion), (b) a 2D interactive built around a predict-then-check gear task, (c) a 3D scene constructing compound gear ratios with teaching panels and an in-scene question, and (d) a quiz with a misconception distractor and a short-answer transfer item.

behavior, which doubles as the retrieval index, so keeping that description current is part of every behavior-changing edit.

Model governance. The platform runs against proxy models named `claude-sonnet-5` (default), `claude-opus-4-8`, and `claude-fable-5` (deep thinker). One operational trap deserves record: with adaptive thinking, the tokens a model spends thinking count against its output budget, and when the budget is too small the model's answer is silently truncated. The symptom was maddeningly quiet: the Planner would return nothing parseable, or the Executor would reply "Done." having done nothing. The cure was governance, not prompting: capture usage and stop reasons on every call, give planning calls a guaranteed floor of thinking room and execution calls a larger one, and automatically retry once with a doubled budget when truncation is detected.

3.5 Learning Companion Design

The platform has two faces: creation and learning. The right-hand agent viewport (described fully in Section 3.7) serves both. During creation it hosts the authoring agent with its three working modes for building scenes, sections, and the learning outline. When the author switches to learning mode, which unlocks once every section of the course has been built, the same viewport becomes a learning companion restricted to conversation only: it can explain, question, and sketch, but it cannot modify the course.

The companion is designed around two ideas. The first is *live learning context*: the companion continuously reads where the learner is, the content of the section currently open, the learner's position in the outline, and the outcomes of recent quiz answers, so its help is grounded in what the learner is actually looking at

rather than in the course in general. Wrong quiz answers reach the companion, which lets it bring up the misconception at the right moment instead of lecturing in the abstract. The second is *step-by-step guidance* in the spirit of Khan Academy’s tutoring assistant [14]: the companion is Socratic by default, preferring questions and hints over answers, walking the learner through the generated material in its intended order, section type by section type, and drawing small diagrams when a picture serves better than a sentence.

3.6 Storage, Samples, and Deployment

Courses are ordinary files. A finished course, its outline, knowledge graph, content, and 3D scenes included, saves as a single portable document that can be stored, shared, and reopened; the platform also keeps a small in-browser library of recent projects and continuously stashes the working draft so that closing the tab loses nothing. Imported courses pass safety gates, including an explicit confirmation before any course containing generated code is allowed to run.

Five pre-built sample courses ship with the platform, one per subject domain: biology (viruses), chemistry (molecular geometry), geoscience (terrain), mechanical engineering (gears), and physics (projectile motion); Appendix A (Table 6) lists them. Samples open instantly from the project screen and were the backbone of our study.

The public demo is hosted on GitHub Pages, which is a static file host rather than a real server. The gap matters: a static host cannot run document conversion, cannot generate images, cannot reach the model provider directly, and cannot keep server-side logs. The platform detects which environment it is in and degrades gracefully, but the gap had real consequences: it is why our study used pre-built samples rather than live document-to-course generation (Section 5), and it bounds what the public demo can show, a limitation we return to in Section 8.

3.7 User Interface

The interface (Figure 4) is a three-pane layout with a utility bar on top.

Left (A): the learning outline. The left pane shows the course outline, chapters and sections with their progress rings. When the active section is a 3D scene, the pane gains three mini-tabs: *Outline* (the default), *Assets*, a palette of 3D assets that can be dragged and dropped straight into the scene, and *Hierarchy*, which shows how the current 3D scene is organized.

Center (B): the modality workspace. The center shows whichever section is active in its own medium: prose for reading sections, the live interactive page for 2D sections, the quiz with its answer flow, or the full 3D viewport with orbit navigation. Figure 3 previews all four. The 3D viewport is never torn down when switching sections, which keeps switching instant, and an edit-versus-play toggle lets the author try a scene as a learner would and then roll back to the exact editing state.

Right (C): the agent viewport. The right pane hosts the agent. During creation it offers the three building modes for scene, section, and outline work; in learning mode it becomes the learning companion of Section 3.5.

Top (D): utilities and global configuration. The top bar collects everything global: project management and sample loading, settings including interface language and font size, save, share, and download.

4 Development Progress and Engineering Lessons

Because this is a course project, we record here the path we actually took: the iterations, the difficulties we hit, how we got through them, and what we learned.

4.1 From a VR Platform to a Learning Experience Builder

This project did not begin as what it is now. We started out building a codeless VR platform: a tool for assembling immersive 3D teaching scenes without programming. The early demos were fun to show off, a miniature solar system among them, and for a while we mistook that for progress. The turning point was an uncomfortable observation from our own pilot: once the novelty of looking at the scene wore off, nothing about it taught. Piling up fancy-looking content was not helping anyone learn; there was no misconception being confronted, no insight being constructed, nothing a student could carry to the next problem.

So we tried to add teaching logic to the scenes. That led to a second realization: much of what good teaching needs (careful reading, quizzes, reflection) has no business being inside a 3D scene at all. Forcing prose onto floating panels in a virtual room is worse than a page. Step by step, the project reshaped itself from “a VR scene builder” into “a builder of complete learning experiences,” where 3D is one modality among four, admitted only when the concept lives in space.

The final push away from VR was practical. When we designed the study, we found we could not guarantee that participants, remote and using their own machines, would have access to VR headsets; testing an experience nobody can open is not a study. So for this iteration we deliberately fell back to desktop 3D with orbit controls and hid the VR entry points rather than delete them; the work is still there for when we want it (Section 8.2). We think of this not as abandoning immersion but as sequencing it after the part that matters more: the pedagogy.

4.2 The Cursor Gap: Affordances, not Intelligence

Early on we hit a puzzle: the very same model that wrote rich, beautiful 3D scenes when used inside a code editor produced crude blocky shapes through our platform’s object-creation command. For a while this felt like the model being lazy. It was not. The difference was the channel we gave it: in an editor the model writes expressive code, while our command asked it to fill in a cramped form. Once we saw that, the fix was obvious in hindsight: give creative work a code path and structural work a deterministic path, which became the dual-track design of Section 3.3. Capability was never the bottleneck; affordance was.

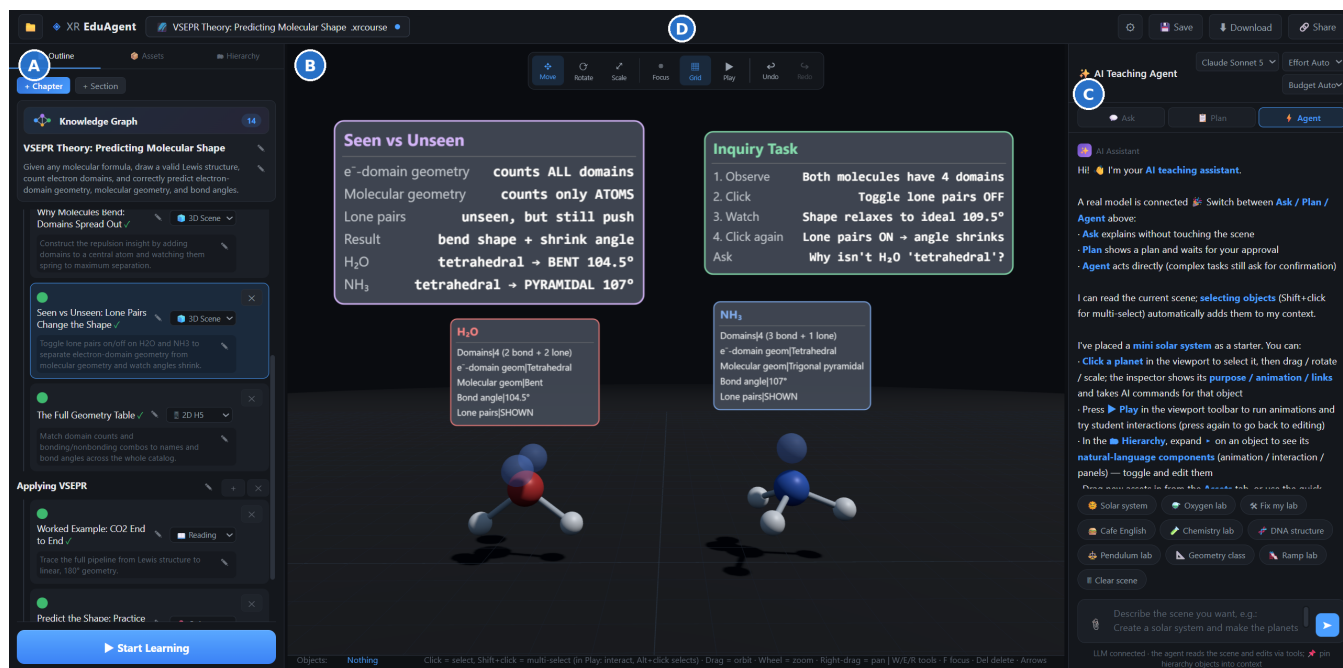


Figure 4: The Keyi authoring interface (molecular-geometry sample course): (A) the learning outline with build-progress rings, (B) the modality workspace showing a live 3D scene, (C) the agent viewport with its Ask, Plan, and Agent modes, and (D) the utility topbar. The screenshot shows the prototype under its working title.

4.3 Flying Blind: The Case for Observability

Our worst stretch of development was not a dramatic crash but a quiet one. For days the agent simply failed without error messages, and we tuned prompts on superstition. Only after we instrumented properly did the real cause surface: the responses were being silently truncated by their own thinking budgets, the trap described in Section 3.4. The budget fix itself was small. The lesson was not: we had been debugging by feel. After that we logged everything the model did, and later problems took hours instead of days.

4.4 Letting the Pipeline Be Rebuilt

As the course generator matured, outline quality kept collapsing: sections wandered off the material, quizzes tested things never taught. We kept patching the material-to-outline flow until the honest conclusion became unavoidable: stop layering compatibility onto a broken pipeline and rebuild it around the pedagogy. That rebuild became the insight-first pipeline this paper describes (Section 3.1), with the checking done by code. It was the single highest-leverage change in the project.

4.5 The Scene Isolation War

The hardest pure engineering arc came from a single architectural fact: there is only one live 3D scene, shared by every 3D section. Everything that could go wrong with a shared mutable resource did. Only the first 3D section would build; later sections became copies of the first under new titles; an ambitious model would invent objects that could not survive saving and wipe out finished work; an innocent chat edit would erase a section's completed status. The

fix, in the end, was boring: every object had to know which section owned it, and nothing was allowed to touch a section it did not belong to. The full set of guards that came out of this fight lives in Section 3.3; each one of them is a scar from a specific bug.

4.6 “It Works on My Machine,” Deployment Edition

Our public demo broke in ways our local build never did. Images that displayed perfectly during development vanished on the public site, because generated courses were referencing files that only existed on the development machine; a fix would appear to work, then break again, because of deployment lag and caching. The remedies were unglamorous: embed or properly package every asset a course references, always verify which build is actually live before judging a fix, and look at the rendered result with your own eyes rather than trusting that a referenced image is the image you think it is. Content operations turned out to be part of the product. Hosting dependence also cost us data: a GitHub outage during the study window took the public demo offline, and the eight to ten sessions we had initially planned had to be cut to six.

4.7 What We Would Tell the Next Team

Five lessons feel transferable beyond this project. Instrument before you tune; you cannot fix what you cannot see. Encode pedagogy and safety as data contracts checked by code, because a long pipeline cannot run on the model's good behavior. Treat every shared mutable resource as a bug family waiting to happen and give it ownership rules early. Listen when your own gut says to rebuild instead of

patch; the rebuild was our best decision. And after any batch of fixes, prove that only the things you meant to change changed.

5 Study Design

5.1 Goals and Scope

We ran an informal usability study. The question is whether the *produced learning experience* (sample courses on the public demo) is understandable and satisfying, and whether it is associated with a learning-gain signal on a domain quiz, plus qualitative feedback for iteration. We initially planned eight to ten sessions, but a hosting outage during the study window cut recruitment short (Section 4); six sessions were completed and analyzed ($n = 6$).

5.2 System Under Test

Sessions use the public demo build with pre-built sample courses matched to the assigned learning pack; samples sidestep the document-conversion dependency that the static host cannot provide (Section 3.6). Participants use desktop Chrome or Edge, with Zoom screen share and cloud recording.

5.3 Materials

Five packs pair a sample course with a pre/post quiz and grading rubric, one per domain, matching the five sample courses of Section 3.6. Each quiz has 15 multiple-choice questions plus 5 short answers scored 0 to 3, for a total of 30 points, and the same instrument is used pre and post.

5.4 Protocol

Each session runs 60 to 90 minutes: greeting and framing; random assignment of one pack (no self-selection); pre-test with downloadable answer sheet; opening the matching sample course and starting the learning mode; a full pass through the course with every section confirmed opened; post-test in a fresh tab; an experience questionnaire; a 5 to 10 minute semi-structured interview; and recording handoff.

5.5 Instruments

Four instruments: (1) the domain pre/post quiz; (2) a questionnaire covering demographics, raw NASA-TLX [12] on six scales from 0 to 100, and 11 Likert items on a 1 to 7 scale, spanning satisfaction, reuse, and recommendation (S1 to S3), the authoring experience such as clarity of flow, waiting, outline clarity, and trust (A1 to A4), and modality-specific learning for reading, quiz, interactive, and 3D (L1 to L4), plus open best and worst questions; (3) the semi-structured interview (tool understanding, waiting, most helpful modality, confusion, depth, willingness to study or teach with it, one requested fix); and (4) an optional timing log. Four interviews were captured as full transcripts; two participants (P5, P6) have interviewer keynotes only.

5.6 Analysis Plan

Multiple-choice items are scored by exact match against the pack’s answer key; short answers are scored 0 to 3 analytically against the rubric’s expected content points, auto-scored first and then adjudicated by a single rater where the keyword heuristic under-scored

Table 2: Participants ($n = 6$), identified only by study IDs.

ID	Pack	Role / field	Age	AI use
P1	Chemistry (VSEPR)	Working, CS	24	Weekly+
P2	Mechanical (gears)	Grad, Data Sci.	24	Weekly+
P3	Physics (projectile)	Grad, CS	24	Weekly+
P4	Biology (virus)	Grad, CS	24	Weekly+
P5	Geoscience (terrain)	Working, Finance	32	Sometimes
P6	Physics (projectile)	Undergrad, Art	18	Weekly+

Table 3: Pre/post quiz scores and gains ($n = 6$). Short-answer (SA) scores include single-rater rubric adjudication. P4’s pre-test was an unscorable image-only PDF, treated as 0; P5’s pre-test was attempted but scored 0.

ID	Pre /30	Post MCQ	Post SA	Post /30	Gain
P1 (VSEPR)	0	15/15	13/15	28	+28
P2 (gears)	0	15/15	12/15	27	+27
P3 (projectile)	0	15/15	11/15	26	+26
P4 (virus)	0	14/15	6/15	20	+20
P5 (terrain)	0	11/15	7/15	18	+18
P6 (projectile)	9	13/15	11/15	24	+15
Mean	1.50	13.83	10.00	23.83	+22.33
(SD, total)	(3.67)			(4.02)	(5.39)

clearly correct answers. Interview material is coded inductively into themes [4]. In aggregate we report NASA-TLX means (noting that the Performance scale is reverse-worded in the interface, with 0 meaning “did great,” and must be flipped when averaging load) and Likert means. We run no inferential statistics at this sample size.

5.7 Participants

Table 2 lists the participants. All five packs were used, with the projectile pack assigned twice. The sample skews toward CS-adjacent graduate students in their twenties with high AI familiarity; P5 (finance, age 32, occasional AI use) and P6 (art undergraduate, age 18) diversify it somewhat, and turn out to be informative outliers in the results.

6 Results

Figure 5 summarizes the quantitative picture below in one view.

6.1 Learning-Gain Signal

Five of the six pre-tests sit at the floor, though not identically: P1 to P3 left the pre-test blank; P4 submitted the pre-test as an image-only PDF from which no answers could be extracted, and stated in the interview that they “did not have any idea” on the first quiz, so we treat it as 0; P5 attempted four multiple-choice items and got all four wrong. P6 is the only informative non-zero baseline, entering at 9 of 30. Table 3 shows the scores.

Four observations. First, the mean gain is large: +22.3 of 30, about three quarters of the scale, driven mainly by multiple choice (from 1.17 to 13.83 of 15 on average, +12.7). Second, post performance is strong but not uniformly at ceiling: three participants hit 15/15 on

Table 4: Raw NASA-TLX responses (0 to 100, $n = 6$). The Performance scale is reverse-worded in the interface (0 means “did great”), so no combined workload index is computed from these raw values.

	Ment.	Phys.	Temp.	Perf*	Eff.	Frus.
P1	75	10	30	10	50	75
P2	80	65	15	90	70	30
P3	75	60	10	80	70	20
P4	80	25	25	50	75	25
P5	40	30	35	60	80	70
P6	30	20	15	75	15	35
Mean	63.3	35.0	21.7	60.8	60.0	42.5

MCQ, and post totals range from 18 to 28. Third, short answers lag multiple choice (mean 10.0 versus 13.8 of 15): participants recognize facts more reliably than they articulate full rubric points, and some skipped short-answer items outright. Fourth, P6, the one participant with a real baseline, still gained +15, moving from 7 to 13 on MCQ; this is the single cleanest learning signal in the data. Descriptively, the terrain and virus packs produced the lowest post totals, but with one or two participants per pack we do not interpret pack differences.

The floor-effect caveat must be stated plainly: with five near-zero baselines, these gains show that participants *could answer after the course*, not a controlled knowledge delta. The same instrument was used pre and post, so practice and memory effects are also possible. We treat the quiz results as a coarse check of the learning experience, nothing more.

6.2 Workload (NASA-TLX)

Table 4 shows the raw responses. Mental demand runs moderately high (mean 63.3), and highest for the CS-adjacent participants working STEM packs (75 to 80), while P5 and P6 sit lower (40 and 30). Temporal demand is low across the board (mean 21.7): waiting was not the burden. Effort is high (mean 60.0) with the striking exception of P6 (15), who also gave ceiling satisfaction. Frustration is bimodal: P1 (75) and P5 (70) versus 20 to 35 for everyone else, and the interviews map these two directly onto the 3D-trust and thin-reading themes below. The Performance column needs caution: P1’s low value is consistent with “did well,” but several high values likely indicate polarity confusion with the reverse-worded slider (flipping the scale yields a mean of 39.2), so we report this scale descriptively rather than folding it into a workload index.

6.3 Satisfaction (Likert)

Table 5 shows every item. Four patterns stand out. The satisfaction cluster is strong: S1 to S3 average 6.44, and recommendation (S3) averages 6.67, with only P4 below 6. Among the modality items, quiz, interactive, and 3D all average 6.50 while reading is the lowest-rated modality at 5.67, pulled down by P5 (4) and P1 (5); the questionnaire agrees with the interviews about where the weak link is. The authoring-experience cluster runs softer than satisfaction (wait 5.33, outline clarity 5.67, trust 5.83), matching the fidelity and interface concerns below. Finally, two participants anchor the extremes: P4

is the dissatisfied outlier (overall satisfaction 4, outline clarity and trust both 4), aligned with their interface complaints, while P6 is a ceiling responder (all 7s), useful as an existence proof of delight but not as an average.

6.4 Open Questionnaire Text

P1 named 3D as the best part, for making abstract concepts easier and letting learners experience otherwise hard-to-access situations, and worried whether the AI truly understands real-world physics and chemistry rules inside the generated 3D world. P2 valued the organized interactive learning but noted that generation sometimes was “not conveying proper information.” P3 wrote that 2D and 3D visualizations made concepts click faster but found the reading too short for trickier topics. P4 named the follow-up quiz as the best part and the interface as the worst. P5 praised the follow-up quiz and the learning companion for checking understanding, but found the reading assumed prior knowledge a novice does not have, which became frustrating. P6 wrote that “any part is good” and reported no frustration at all.

6.5 Interview Themes

Eight themes recur across the six interviews.

(1) *The tool is understood as an automatic multimodal course builder* (P1 to P4, P6): “builds a whole course by itself, 3D scenes, readings, little interactives, quizzes, with an AI guide” (P2).

(2) *Waiting is acceptable* (five of six): “pretty normal and acceptable” (P1); “a bit slower than I expected at first” but “wasn’t really a dealbreaker” (P2); “small wait but nothing that bothered me ... pretty reasonable” (P3).

(3) *Visualization and interaction are the perceived learning engine* (P1, P3, P4, P6): “the 3D helps the most because it makes the abstract concept easier to understand” (P1); “seeing the concept actually laid out made it click faster than reading” (P3); “wish I could have such a system when I was high school” (P6).

(4) *The quiz and companion loop forces real checking* (P2, P4, P5): “the quiz forced me to actually check if I understood, instead of just skimming” (P2); P5 valued mistakes showing *how* understanding failed.

(5) *Reading is thin and under-scaffolded* (P1, P3, P5): only “two or three sentence[s] to introduce one concept” (P1); P5, the one true domain novice, could not follow later chunks because the reading assumed prior knowledge, and asked for either richer background or a way to pick a starting level.

(6) *3D trust is fragile* (P1, P2, mildly P3): scenes that would “load half broken” where “some objects just wouldn’t show up right” shake trust (P2); P1 doubted the AI’s grasp of real-world physical rules; P3 noticed minor scene settle-in glitches.

(7) *Depth reads as light but acceptable* (P2 to P5): “mostly about right, but leaning a bit on the lighter side” (P3); “didn’t go as deep” (P4).

(8) *Study yes, teach maybe, and one fix each*: nearly all would use it for their own studying (P4 conditioned this on the material), and P6 would recommend it for high school outright (“will go viral”), while P5 would not recommend it to friends until the reading serves novices. The requested fixes span a more step-by-step explanatory tutor (P1), trustworthy 3D rendering (P2), more text for harder

Table 5: All 11 Likert items (1 to 7, $n = 6$). S = satisfaction cluster (S1 overall, S2 reuse, S3 recommend); A = authoring experience (A1 clarity of flow, A2 wait acceptable, A3 outline clear, A4 trust content); L = modality helpfulness (L1 reading, L2 quiz, L3 interactive, L4 3D).

ID	S1	S2	S3	A1	A2	A3	A4	L1	L2	L3	L4	S1 to S3	All 11
P1	6	7	7	6	6	5	5	5	6	7	6	6.67	6.00
P2	7	6	7	5	5	6	7	6	6	6	6	6.67	6.09
P3	6	6	7	5	4	6	6	6	7	6	7	6.33	6.00
P4	4	6	5	6	5	4	4	6	6	6	6	5.00	5.27
P5	7	7	7	5	5	6	6	4	7	7	7	7.00	6.18
P6	7	7	7	7	7	7	7	7	7	7	7	7.00	7.00
Mean	6.17	6.50	6.67	5.67	5.33	5.67	5.83	5.67	6.50	6.50	6.50	6.44	6.09

topics (P3), a less congested interface with a focused study view (P4), a prior-knowledge on-ramp (P5), and document upload on the public site (P6).

7 Analysis and Discussion

7.1 What the Evidence Supports

Three lines of evidence converge. First, the courses teach something measurable: the mean gain is +22.3 of 30 with the floor-effect caveat, and P6, the one participant with a genuine non-zero baseline, still gained +15. Second, the multimodal construct is the perceived learning engine: quiz, interactive, and 3D items all average 6.50 of 7 while interviews independently name visualization and quizzes as what made concepts land, which is what the scaffold, construct, transfer design predicts. Third, satisfaction survives workload: mental demand averages 63 yet satisfaction averages 6.44 and recommendation 6.67, so effortful learning did not read as a bad product; frustration, where it appeared, tracked trust and reading problems rather than difficulty itself. Temporal load is low throughout, so waiting is not the bottleneck when samples are used.

7.2 What the Evidence Challenges

Five findings push back on the design. Reading as a scaffold is under-delivered: it is the lowest-rated modality (5.67), three participants called it thin, and the one true novice (P5) could not attach later material to missing background, exposing the absence of a prior-knowledge on-ramp. Trust in the physical fidelity of AI-generated 3D remains fragile, and half-broken scenes are not cosmetic problems: they undermine the constructivist loop the scenes exist to serve. The gap between recognition and articulation (post MCQ 13.8 versus SA 10.0) suggests the courses currently install recognition more reliably than the rehearsable, transferable statements the aha keys aim for. The interface itself cost us our one dissatisfied participant (P4), whose complaints centered on congestion rather than content. Finally, the NASA-TLX Performance polarity needs clearer wording or explicit instruction before any workload profile can be claimed.

7.3 Design Implications

The findings map onto a concrete backlog. Thin reading calls for a minimum chunk depth, an explicit misconception-naming requirement in generation, and a prior-knowledge gateway that lets a learner pick a starting level or receive a background capsule first; text cohesion and structure are measurable properties with known

effects on comprehension [10, 19], so reading quality can become a checked contract like everything else in the pipeline. Fragile 3D trust calls for hardened scene distinctness, runtime smoke tests, and a scene-health surface for authors. The recognition-versus-articulation gap argues for more generative output in the loop: having learners state the insight in their own words, with the companion checking it. The tutor requests argue for expanding the companion toward guided step-by-step explanation while staying Socratic. The interface needs a focused study view that hides authoring chrome. And P6’s request for uploads on the public demo is really a deployment request: closing the static-host gap of Section 3.6.

7.4 Connecting Back to the Thesis

The study exercises the learner half of the thesis: installing keys through multimodal sections. The authoring half (the insight, knowledge graph, outline loop) is evidenced mainly by the system design and the iteration history, not by these sessions, which opened pre-built courses. A follow-up should also measure what the OECD evidence says matters most: retention with the AI switched off [2, 22], which our companion-assisted post-test does not isolate. Author-facing sessions and an unassisted delayed post-test are the two clearest next steps.

8 Limitations and Future Work

8.1 Limitations

The study is small ($n = 6$, reduced from a planned eight to ten by the hosting outage described in Section 4) and uses a convenience sample that skews toward CS-adjacent graduate students with high AI familiarity. There is no control condition and no delayed post-test, five of six pre-tests sit at the floor (one of them an unscorable image-only submission), and the same instrument was used pre and post, so practice effects cannot be ruled out. Short-answer scoring mixes automation with single-rater adjudication rather than double-blind coding, and two of the six interviews survive only as keynotes rather than transcripts. The study used pre-built samples rather than live document-to-course generation, a constraint inherited from the static public host (Section 3.6), so external validity for the upload-and-build experience differs. Participants viewed 3D through desktop orbit controls, a deliberate fallback described in Section 4. On the system side: there is no runtime mastery model of the learner yet; the planned pattern library and a critic that cross-checks generated content against the knowledge graph are

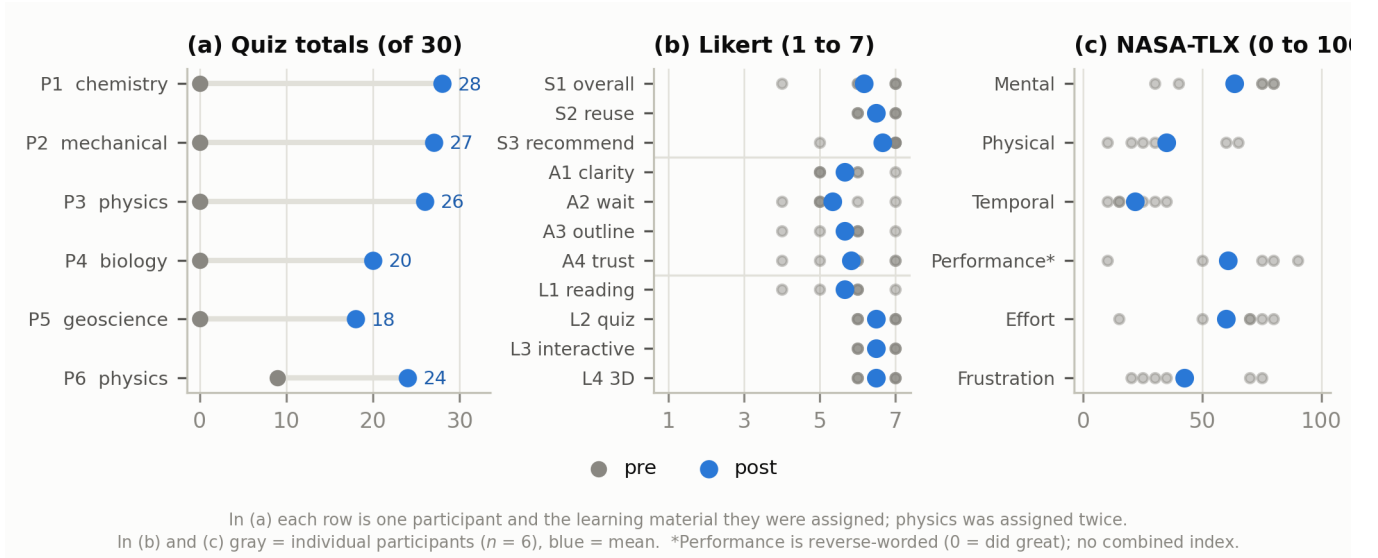


Figure 5: Study results overview ($n = 6$): (a) per-participant quiz totals from pre to post, with post scores labeled; (b) all 11 Likert items, individual responses in gray and means in blue, grouped as satisfaction, authoring experience, and modality helpfulness; (c) raw NASA-TLX responses with the reverse-worded Performance scale flagged.

only partially wired, with the current version relying on spatiality heuristics; generated-object code runs in the page rather than in stronger isolation; and the public demo inherits the other static-host constraints of Section 3.6, including the inability to keep access keys private.

8.2 Future Work

On the study side, we will add author-facing sessions, a delayed post-test taken without the companion (the outcome the field’s strongest evidence says to watch [2, 22]), and a broader sample beyond CS-adjacent graduate students. On the pedagogy engine, we will wire the pattern-lookup engine and the graph-checking critic so that concepts used but never taught are caught mechanically, add mastery measurement at the level of individual insights, strengthen the reading scaffolds with a prior-knowledge on-ramp, and make the companion’s guidance more conversational. On the platform, we plan a focused study view, document upload on the hosted demo, stronger isolation for generated code, imported standard 3D model formats, an asset-reuse loop in which well-built objects become reusable skills, accounts and cloud saving with community sharing, and speech input and output.

One direction deserves its own mention. The current 3D runtime was built so that its skills and tools can extend to immersive delivery: through the WebXR plug-in path, the same generated scenes can become VR-compatible without rearchitecting. We deliberately deferred this (Section 4), but we consider it valuable for particular learning contexts where embodiment is the point, such as ESL dialogue speaking practice, and we will detail this direction in future work.

9 Conclusion

Keyi attacks a concrete failure mode of LLM course generation: pretty content dumps. By distilling the insights a course must install before any content exists, anchoring them in a checkable knowledge graph, holding every section accountable to what it covers and installs, and surrounding the agent with dynamically loaded skills, disciplined tools, and a strictly isolated 3D build process, the system aims to compile teaching materials into experiences that install transferable keys. A completed remote study ($n = 6$) found large quiz gains (mean +22.3 of 30, with floor-effect caveats), strong satisfaction (6.44 of 7), and consistent qualitative agreement that the interactive modalities and quizzes carry the learning, while pointing clearly at reading scaffolds, a prior-knowledge on-ramp, and 3D trust as the next engineering targets. The portable claim is methodological: pedagogy and stability both become contracts, not prompts alone. A course is done when the learner *keyi*: when they are able.

References

- [1] Christoph Auer, Maksym Lysak, Ahmed Nassar, Michele Dolfi, Nikolaos Livathinos, Panos Vagenas, Cesar Ramis Cabello, Macey Morin, Roberto Del Valle, Rafael Teixeira de Lima, and Peter Staar. 2024. Docling Technical Report. arXiv:2408.09869.
- [2] Hamsa Bastani, Osbert Bastani, Alp Sungu, Haosen Ge, Özge Kabakcı, and Rei Mariman. 2025. Generative AI Without Guardrails Can Harm Learning: Evidence from High School Mathematics. *Proceedings of the National Academy of Sciences* 122, 26 (2025), e2422633122. doi:10.1073/pnas.2422633122
- [3] Elizabeth L. Bjork and Robert A. Bjork. 2011. Making Things Hard on Yourself, but in a Good Way: Creating Desirable Difficulties to Enhance Learning. In *Psychology and the Real World: Essays Illustrating Fundamental Contributions to Society*. Worth Publishers, New York, NY, 56–64.
- [4] Virginia Braun and Victoria Clarke. 2006. Using Thematic Analysis in Psychology. *Qualitative Research in Psychology* 3, 2 (2006), 77–101.
- [5] Michelene T. H. Chi and Ruth Wylie. 2014. The ICAP Framework: Linking Cognitive Engagement to Active Learning Outcomes. *Educational Psychologist* 49, 4 (2014), 219–243.

- [6] Richard E. Clark. 1983. Reconsidering Research on Learning from Media. *Review of Educational Research* 53, 4 (1983), 445–459.
- [7] Albert T. Corbett and John R. Anderson. 1995. Knowledge Tracing: Modeling the Acquisition of Procedural Knowledge. *User Modeling and User-Adapted Interaction* 4, 4 (1995), 253–278.
- [8] Fernanda De La Torre, Cathy Mengying Fang, Han Huang, Andrzej Banburski-Fahey, Judith Amores Fernandez, and Jaron Lanier. 2024. LLMR: Real-Time Prompting of Interactive Worlds Using Large Language Models. arXiv:2309.12276.
- [9] Arthur C. Graesser, Shulan Lu, George T. Jackson, Heather H. Mitchell, Mathew Ventura, Andrew Olney, and Max M. Louwerse. 2004. AutoTutor: A Tutor with Dialogue in Natural Language. *Behavior Research Methods, Instruments, & Computers* 36, 2 (2004), 180–192.
- [10] Arthur C. Graesser, Danielle S. McNamara, Max M. Louwerse, and Zhiqiang Cai. 2004. Coh-Matrix: Analysis of Text on Cohesion and Language. *Behavior Research Methods, Instruments, & Computers* 36, 2 (2004), 193–202. doi:10.3758/BF03195564
- [11] Kelvin Guu, John Miller, and Percy Liang. 2015. Traversing Knowledge Graphs in Vector Space. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing (EMNLP 2015)*. Association for Computational Linguistics, Lisbon, Portugal, 318–327. doi:10.18653/v1/D15-1038
- [12] Sandra G. Hart and Lowell E. Staveland. 1988. Development of NASA-TLX (Task Load Index): Results of Empirical and Theoretical Research. In *Human Mental Workload*, Peter A. Hancock and Najmedin Meshkati (Eds.). Advances in Psychology, Vol. 52. North-Holland, Amsterdam, 139–183.
- [13] Gregory Kestin, Kelly Miller, Anna Klaes, Timothy Milbourne, and Gregorio Ponti. 2025. AI Tutoring Outperforms In-Class Active Learning: An RCT Introducing a Novel Research-Based Design in an Authentic Educational Setting. *Scientific Reports* 15 (2025). doi:10.1038/s41598-025-97652-6
- [14] Khan Academy. 2023. Khanmigo: AI for Education. <https://www.khanmigo.ai/>. Accessed 2026-08.
- [15] Robert B. Kozma. 1994. Will Media Influence Learning? Reframing the Debate. *Educational Technology Research and Development* 42, 2 (1994), 7–19.
- [16] LearnLM Team et al. 2025. LearnLM: Improving Gemini for Learning. arXiv:2412.16429.
- [17] LearnLM Team and Google. 2025. Towards an AI-Augmented Textbook. arXiv:2509.13348.
- [18] Chen Liang, Zhaohui Wu, Wenyi Huang, and C. Lee Giles. 2015. Measuring Prerequisite Relations Among Concepts. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing (EMNLP 2015)*. Association for Computational Linguistics, Lisbon, Portugal, 1668–1674. doi:10.18653/v1/D15-1193
- [19] Erin J. Lightman, Philip M. McCarthy, David F. Dufty, and Danielle S. McNamara. 2007. Cohesion and Structural Organization in High School Texts. In *Proceedings of the Twentieth International Florida Artificial Intelligence Research Society Conference (FLAIRS 2007)*. AAAI Press.
- [20] Richard E. Mayer. 2009. *Multimedia Learning* (2nd ed.). Cambridge University Press, Cambridge, UK.
- [21] Joseph D. Novak and Alberto J. Cañas. 2008. *The Theory Underlying Concept Maps and How to Construct and Use Them*. Technical Report IHMC CmapTools 2006-01 Rev 01-2008. Florida Institute for Human and Machine Cognition.
- [22] OECD. 2026. *OECD Digital Education Outlook 2026: Exploring Effective Uses of Generative AI in Education*. OECD Publishing, Paris. doi:10.1787/062a7394-en
- [23] Seymour Papert. 1980. *Mindstorms: Children, Computers, and Powerful Ideas*. Basic Books, New York, NY.
- [24] Jocelyn Parong and Richard E. Mayer. 2018. Learning Science in Immersive Virtual Reality. *Journal of Educational Psychology* 110, 6 (2018), 785–797.
- [25] Chris Piech, Jonathan Bassen, Jonathan Huang, Surya Ganguli, Mehran Sahami, Leonidas J. Guibas, and Jascha Sohl-Dickstein. 2015. Deep Knowledge Tracing. In *Advances in Neural Information Processing Systems 28 (NeurIPS 2015)*.
- [26] Romain Puech, Jakub Macina, Julia Chatain, Mrinmaya Sachan, and Manu Kapur. 2025. Towards the Pedagogical Steering of Large Language Models for Tutoring: A Case Study with Modeling Productive Failure. In *Findings of the Association for Computational Linguistics: ACL 2025*. Association for Computational Linguistics, Vienna, Austria, 26291–26311. doi:10.18653/v1/2025.findings-acl.1348
- [27] Henry L. Roediger and Jeffrey D. Karpicke. 2006. Test-Enhanced Learning: Taking Memory Tests Improves Long-Term Retention. *Psychological Science* 17, 3 (2006), 249–255.
- [28] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*.
- [29] Robin Schmucker, Meng Xia, Amos Azaria, and Tom Mitchell. 2024. Ruffle&Riley: Insights from Designing and Evaluating a Large Language Model-Based Conversational Tutoring System. In *Artificial Intelligence in Education: 25th International Conference (AIED 2024) (Lecture Notes in Computer Science)*. Springer, 75–90. doi:10.1007/978-3-031-64302-6_6
- [30] Noah L. Schroeder and Aubrey C. Kucera. 2022. Refutation Text Facilitates Learning: A Meta-Analysis of Between-Subjects Experiments. *Educational Psychology Review* 34, 2 (2022), 957–987. doi:10.1007/s10648-021-09656-z
- [31] John Sweller. 1988. Cognitive Load During Problem Solving: Effects on Learning. *Cognitive Science* 12, 2 (1988), 257–285.
- [32] Anais Tack and Chris Piech. 2022. The AI Teacher Test: Measuring the Pedagogical Ability of Blender and GPT-3 in Educational Dialogues. In *Proceedings of the 15th International Conference on Educational Data Mining (EDM 2022)*.
- [33] Kurt VanLehn. 2011. The Relative Effectiveness of Human Tutoring, Intelligent Tutoring Systems, and Other Tutoring Systems. *Educational Psychologist* 46, 4 (2011), 197–221.
- [34] Lev S. Vygotsky. 1978. *Mind in Society: The Development of Higher Psychological Processes*. Harvard University Press, Cambridge, MA.
- [35] Rose E. Wang, Ana T. Ribeiro, Carly D. Robinson, Susanna Loeb, and Dorottya Demszky. 2024. Tutor CoPilot: A Human-AI Approach for Scaling Real-Time Expertise. arXiv:2410.03017.
- [36] David Wood, Jerome S. Bruner, and Gail Ross. 1976. The Role of Tutoring in Problem Solving. *Journal of Child Psychology and Psychiatry* 17, 2 (1976), 89–100.
- [37] Yue Yang, Fan-Yun Sun, Luca Weihs, Eli VanderBilt, Alvaro Herrasti, Winson Han, Jiajun Wu, Nick Haber, Ranjay Krishna, Lingjie Liu, Chris Callison-Burch, Mark Yatskar, Aniruddha Kembhavi, and Christopher Clark. 2024. Holodeck: Language Guided Generation of 3D Embodied AI Environments. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 16227–16237.
- [38] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR 2023)*.

A Sample Courses

The demo path is Projects, then Sample courses, then Start Learning. The live demo is at <https://garythekid.github.io/3d-vr-codeless-platform/>. Source code: <https://github.com/GaryTheKid/3d-vr-codeless-platform>. An interactive visualization of the agent pipeline ships in the repository as `agent-pipeline.html`.

Table 6: The five pre-built sample courses.

id	Title (short)	Subject	Sections
bio-virus	Viruses: Structure & Classification	Biology	7
chem-vsepr	VSEPR: Predicting Molecular Shape	Chemistry	7
geo-terrain	The Geologic Story of Zion	Geoscience	6
mecha-gear	Gears: Building Drive Trains	Mechanical	7
phys-projectile	Projectile Motion	Physics	8